

2.

## File handling in Python



File handling is an important part of any application.

Python has several functions for creating, reading, updating, and deleting files.

تعد معالجة الملفات جزءاً مهماً من أي تطبيق.

لدى Python العديد من الدوال لإنشاء الملفات وقراءتها وتحديثها وحذفها.



```
1 The key function for working with files in Python is the open() function.
2 The open() function takes two parameters; filename, and mode.
3 There are four different methods (modes) for opening a file:
4
5
6 "r" - Read - Default value. Opens a file for reading, error if the file does not
7       exist
8 "a" - Append - Opens a file for appending, creates the file if it does not exist
9 "w" - Write - Opens a file for writing, creates the file if it does not exist
10 "x" - Create - Creates the specified file, returns an error if the file exists
11
12 In addition, you can specify if the file should be handled as binary or text mode
13 "t" - Text - Default value. Text mode
14 "b" - Binary - Binary mode (e.g. images)
```

| Filename   | Mode | Reading Method |
|------------|------|----------------|
| "test.txt" | "rt" | read()         |

Code:

```
2_01_openFile_read.py > ...  
1  myFile = open("test1.txt", 'rb')  
2  print(myFile.read())  
3  myFile.close()  
4  myFile.read()
```

Because "r" for read and "t" for text are the default values, you don't need to specify them.

```
1
2   Read a file:
3
4   - The file is in the same directory
5   - The file is in a different directory
6   - The file doesn't exist
7   - Read and print it on screen
8
9
10
11
12
13
14
```

# Complete path

## Code:

```
2_04_openFile_read.py > ...  
1  f = open("D://myfiles/welcome.txt", "r")  
2  print(f.read())  
3
```

Slash instead of back slash

# Read part of a file

Filename

"test.txt"

Mode

"rt"

Reading Method

Read(n)

## Code:

 2\_02\_openFile\_read.py > ...

```
1  # Return the 6 first characters of the file
2  f = open("test.txt", "r")
3  print(f.read(6))
```

1  
2  
3     Read an image file

- 4
- 5     - Al-Aqsa.jpg     (in the same directory)
  - 6     - Binary file
  - 7     - Read and print it on screen
  - 8     - Show the image
  - 9     - Use a library to manipulate the image file
- 10  
11  
12  
13  
14



# Read part of a file (lines)

| Filename   | Mode | Reading Method |
|------------|------|----------------|
| "test.txt" | "rt" | Readline(n)    |

Code:

2\_03\_openFile\_readline.py > ...

```
1  myFile = open("test.txt", 'rt')
2  print(myFile.readline())
3  print(myFile.readline())
4  print(myFile.readline())
5  for x in myFile:
6      | print(x, end='')
7  myFile.close()
```

1 To write to an existing file, you must add a parameter to  
2 the `open()` function:

3 "a" - Append - will append to the end of the file

4 "w" - Write - will overwrite any existing content

Code: 2\_05\_writeToFile.py > ...

```
5 1 f = open("test.txt", "a")
6 2 f.write("\nNow the file has more content!")
7 3 f.close()
8 4 #open and read the file after the appending:
9 5 f = open("test.txt", "r")
10 6 print(f.read())
11 7
12 8 f = open("file1.txt", "w")
13 9 f.write("Woops! I have deleted the content!")
14 10 f.close()
15 11
16 12 #open and read the file after the overwriting:
17 13 f = open("demofile3.txt", "r")
18 14 print(f.read())
```

1 To create a file, you must add a parameter to the `open()` function:  
2 "x" - Create a new file  
3 "w" - Create a new file id it doesn't exist

7 Code:  2\_06\_createFile.py > ...

```
1 f = open("file1.txt", "x") # Create a new file
2 f = open("file2.txt", "w") # Create a new file if it doesn't exist
```

1 To delete a file, you must use the `os` library

2

3 `os.remove(file.txt)` - Remove a file

3

4 `os.rmdir(folder1)` - Remove a directory, you can only delete an empty one

4

5 Code:

2\_07\_deleteFile.py > ...

6

1 `import os`

7

2 `os.remove("file2.txt")`

8

3

9

4 `if os.path.exists("file3.txt"):`

10

5 | `os.remove("file3.txt")`

11

6 `else:`

12

7 | `print("The file does not exist")`

13

14

# Writing to a file

Filename

"test.txt"

Mode

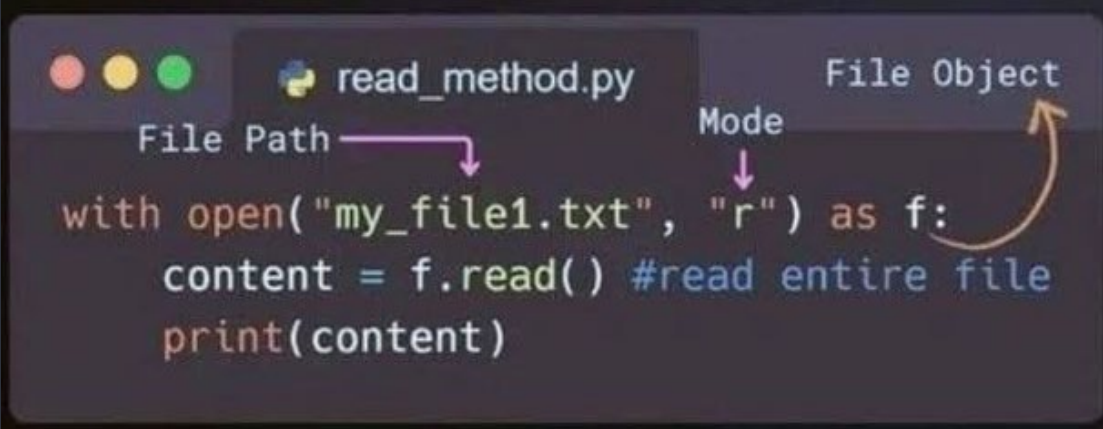
"rt"

Reading Method

read()

Code:

```
2_1_openFile_read.py > ...  
1  myFile = open("test.txt", 'rt')  
2  print(myFile.read())
```

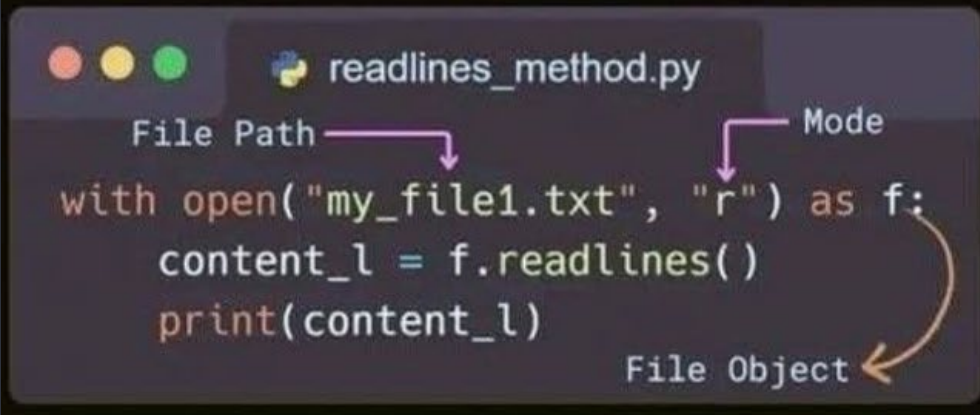


```
1  read_method.py
2
3  File Path
4
5  with open("my_file1.txt", "r") as f:
6      content = f.read() #read entire file
7      print(content)
```

The `read()` method reads a specified number of characters (or bytes) from a file, or the entire file content if no size is specified.

The `readline()` method in Python is employed to sequentially read one line at a time from a file.

```
readline_method.py  
File Path  
with open("my_file1.txt", "r") as f:  
    line1 = f.readline()  
    print(line1)  
Mode  
File Object
```



```
1  readlines_method.py
2
3  File Path  Mode
4  with open("my_file1.txt", "r") as f:
5      content_l = f.readlines()
6      print(content_l)
7
8  File Object
```

The `readlines()` method reads all lines from a file, returning them as a list where each element corresponds to a line in the file.



Code:

```
3_02_errorHandling.py > ...  
1 # bad practice  
2 def read_file(file_path):  
3     file = open(file_path, 'r')  
4     content = file.read()  
5     file.close()  
6     return content  
7 print(read_file('textError.txt'))  
8  
9 # good practice  
10 def read_file(file_path):  
11     try:  
12         with open(file_path, 'r') as file:  
13             return file.read()  
14     except FileNotFoundError:  
15         return "File not found."  
16 print(read_file('text_Error.txt'))
```